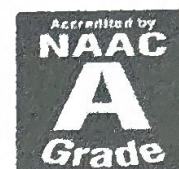


SAVEETHA

INSTITUTE OF MEDICAL AND TECHNICAL SCIENCES
(Declared as Deemed to be University under Section 3 of UGC Act 1956)



SIMATS ENGINEERING ASSESSMENT TOOL 3

COURSE CODE: CSA1407

COURSE NAME: Compiler Design

COURSE OUTCOME COVERED

CO2: Construct top-down and bottom up parsers using CFG for parsing conflicts and error recovery for efficient syntax tree generation. (BL3)

Assessment Tool Weightage: 10%

QUESTIONS:

Total Marks:50

Annexure A

SCENARIO BASED TASK

Code of Conduct:

I Juhi Surin (192525148) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

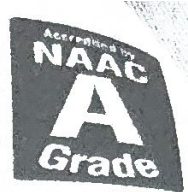
Signature of the student:

Juhi Surin



SAVEETHA

INSTITUTE OF MEDICAL AND TECHNICAL SCIENCES
(Declared as Deemed to be University under Section 3 of UGC Act 1956)



Annexure A

SCENARIO-BASED TASK

QUESTIONS:

- 1.Scenario: Parsing Table Conflict in LL(1) While designing an LL(1) parser, a student constructs a parsing table and finds multiple entries in a single cell for a non-terminal and input symbol combination. This leads to confusion during parsing. Analyze how the syntax analyzer uses FIRST and FOLLOW sets in this process, identify the reason for the conflict, and explain how it affects deterministic parsing. Conclude by suggesting what decision should be taken to modify the grammar so that it becomes suitable for LL(1) parsing.
- 2.Scenario: Incorrect Parse Tree Generation A syntax analyzer generates a parse tree for the expression $id + id - id$, but the evaluation gives incorrect results due to improper grouping of operators. Analyze how the syntax analyzer constructs the parse tree, identify the issue related to associativity, and explain how grammar rules influence the structure of the tree. Finally, discuss what decision should be taken to enforce correct associativity and ensure accurate parsing.
- 3.Scenario: Invalid Expression Handling A syntax analyzer in a compiler receives the token sequence corresponding to the expression $a + * b$ from the lexical analyzer. Although all tokens are valid individually, the parser fails to generate a parse tree and reports an error. In this situation, analyze how the syntax analyzer identifies the error based on grammar rules, determine the key issue in the token arrangement, and explain how parsing techniques such as predictive or shift-reduce parsing handle such cases. Further, discuss what decision should be taken to improve error detection and recovery while maintaining clarity in parsing.
4. Scenario: Ambiguity in Conditional Statements A syntax analyzer is designed using the grammar $S \rightarrow \text{if } E \text{ then } S \mid \text{if } E \text{ then } S \text{ else } S \mid a$. While constructing the parsing table, conflicts arise, and the parser cannot decide which production to apply for certain inputs. In this scenario, explain how the syntax analyzer encounters ambiguity, identify the key issue known as the dangling else problem, and discuss how it impacts parsing. Suggest an appropriate modification or parsing strategy to resolve the ambiguity and ensure correct association of else clauses.
- 5.Scenario: Operator Precedence Conflict A parser is designed for arithmetic expressions using the grammar $E \rightarrow E + E \mid E * E \mid id$. While parsing the input $id + id * id$, the syntax analyzer produces an incorrect parse tree due to ambiguity in operator precedence. Examine how the syntax analyzer processes this input, identify the issue caused by the grammar, and explain how precedence and associativity rules should be applied. Based on this, suggest what changes must be made to the grammar or parsing technique to ensure correct evaluation.

RUBRICS FOR CALCULATION:

Scenario based assessment

Criteria	Weight age (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Understanding of Scenario	20%	Demonstrates clear and complete understanding of the scenario and context	Shows good understanding with minor gaps	Partial understanding; misses key aspects	Misinterprets or fails to understand the scenario
Identification of Key Issues	20%	Accurately identifies all relevant issues and factors involved	Identifies most key issues with minor omissions	Identifies limited issues; some irrelevant points	Unable to identify key issues
Application of Concepts	25%	Applies appropriate concepts effectively to address the scenario	Applies concepts with minor errors	Limited or partially correct application	Unable to apply relevant concepts
Decision Making	20%	Makes well-reasoned, logical decisions with strong rationale	Makes reasonable decisions with some justification	Makes weak decisions with limited justification	No clear decisions made or rationale provided
Clarity of Response	15%	Response is clear, structured, and logically presented	Generally clear with minor issues	Somewhat unclear or poorly structured	Disorganized and difficult to understand

★ SCENARIO BASED TASK ANSWERS ★

1) Scenario : Parsing Table Conflict in LL(1)

In an LL(1) parser, the syntax analyzer constructs the parsing table using the FIRST and FOLLOW sets of grammar symbols. For each production, the parser enters the production in the table according to the terminals in its FIRST set. If a production derives ϵ (epsilon), the parser also uses the FOLLOW set of the non-terminal.

A conflict occurs when more than one production is placed in the same table cell. This happens because :

- The FIRST sets of two productions overlap.
- A production deriving ϵ has its FOLLOW set overlapping with another production FIRST set.
- The grammar is ambiguous, left-recursive or requires more than one symbol of lookahead.

As a result, the parser cannot uniquely decide which production to apply, making the grammar non-deterministic and unsuitable for LL(1) parsing.

Decision: The grammar should be modified by:

- Eliminating left recursion.
- Performing left factoring.
- Ensuring the FIRST and FOLLOW sets do not create conflicts.

After these modifications, each parsing table entry will contain only one production, making the grammar suitable for deterministic LL(1) parsing.

2) Scenario : Incorrect Parse Tree Generation
The syntax analyzer constructs a parse tree according to the grammar rules. For the expression:

$id + id - id$

the grammar determines how operators are grouped. If the grammar does not enforce associativity, the parser may incorrectly group the expression as:

$id + (id - id)$

instead of the correct left - associative grouping:

$(id + id) - id$

This incorrect grouping changes the evaluation order and may produce wrong results.

Grammar rules directly determine the structure of the parse tree. Left - recursive grammar naturally enforces left associativity, while right - recursive grammar enforces right associativity.

Decision: Use grammar rules that explicitly enforce left associativity, for example:

$E \rightarrow E + T \mid E - T \mid T$

$T \rightarrow id$

or use parse precedence and associativity declarations. This ensures the parser generates the correct parse tree and evaluates expressions accurately.

3>

Scenario : Invalid Expression Handling

The lexical analyzer correctly recognizes the tokens:

$a + * b$

Although each token is valid individually, the sequence does not follow the grammar rules for arithmetic expressions.

After the '+' operator, the grammar expects an operand, but another operator ('*') appears instead.

Therefore, the syntax analyzer cannot construct a valid parse tree and reports a syntax error.

In predictive parsing, the parser checks the parsing table and finds no valid production for the current input symbol, leading to an error.

In shift-reduce parsing, the parser cannot perform either a valid shift or reduction according to the grammar, so it reports a syntax error.

Decision :

To improve error handling :

- Use meaningful syntax error messages.
- Apply error recovery techniques such as panic mode recovery or phase-level recovery.
- Continue parsing after recovery whenever possible.

These techniques improve error detection while maintaining parsing clarity.

4) Ambiguity in Conditional Statements

Given the grammar:

$S \rightarrow \text{if } E \text{ then } S$
 $\mid \text{if } E \text{ then } S \text{ else } S$
 $\mid a$

the syntax analyzer encounters ambiguity because it cannot determine whether an else belongs to the nearest if or an earlier if.

This is known as the Dangling Else Problem.

Due to this ambiguity:

- Multiple parse trees are possible.
- Parsing table conflicts occur.
- Deterministic parsing becomes impossible.

Decision:

The ambiguity can be removed by rewriting the grammar using matched and unmatched statements, for example:

Matched $\rightarrow \text{if } E \text{ then Matched else Matched } \mid a$

Unmatched $\rightarrow \text{if } E \text{ then } S$

$\mid \text{if } E \text{ then Matched else Unmatched}$

Alternatively, use a parser strategy that always associates else with the nearest unmatched if, which is the standard rule used by most programming languages.

> Operator Precedence Conflict

Consider the grammar:

$$\begin{aligned} E &\rightarrow E + E \\ &\quad | E * E \\ &\quad | id \end{aligned}$$

This grammar is ambiguous because it does not specify operator precedence or associativity.

For the expression:

$id + id * id$

the parser may generate different parse trees, such as:

- $(id + id) * id$
- $id + (id * id)$

The correct interpretation is:

$id + (id * id)$

because multiplication has higher precedence than addition.

The ambiguity cause incorrect evaluation.

Decision:

Modify the grammar to explicitly define precedence and associativity, for example:

$$\begin{aligned} E &\rightarrow E + T \mid T \\ T &\rightarrow T * F \mid F \\ F &\rightarrow id \end{aligned}$$

Here:

- $*$ has higher precedence than $+$.
- Both operators follow left associativity.

Alternatively, use parser precedence and associativity rules in LR parsers to resolve conflicts.